SpecUtils Update

Nicholas Earl

Space Telescope Science Institute

specutils

An astropy package for spectroscopy

The `specutils` package provides a basic interface for the loading, manipulation, and common forms of analysis of spectroscopic data. These generic data containers and accompanying modules will provide a toolbox that the astronomical community can use to build more domain-specific packages. For more details about the underlying principles, see APE13, the guiding document for spectroscopic development in the Astropy Project.

Note

While specutils is available for general use, the API is in an early enough development stage that some interfaces may change if user feedback and experience warrants it.

Getting started with specutils

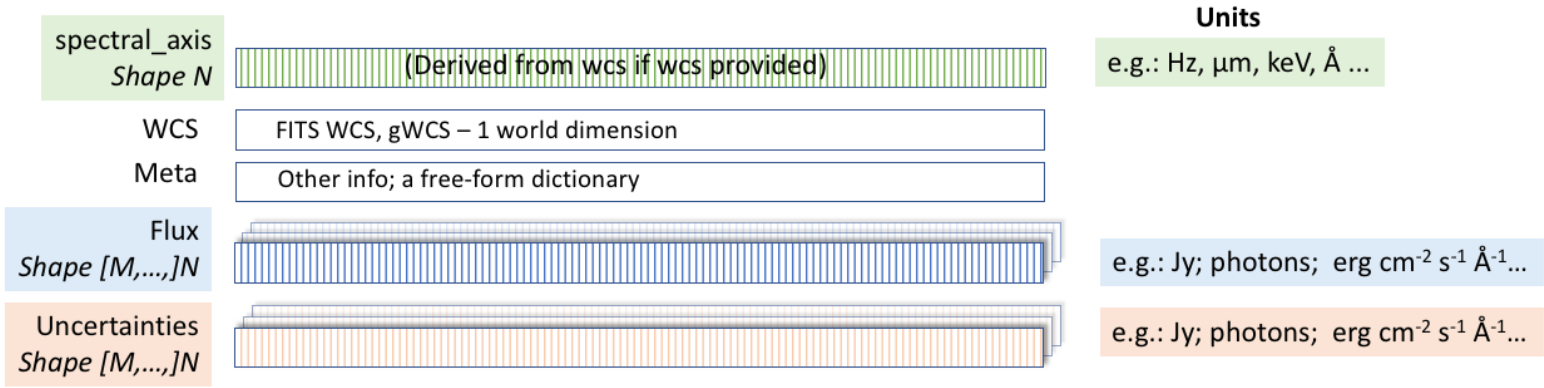
As a basic example, consider an emission line galaxy spectrum from the SDSS. We will use this as a proxy for a spectrum you may have downloaded from some archive, or reduced from your own observations.

We begin with some basic imports:

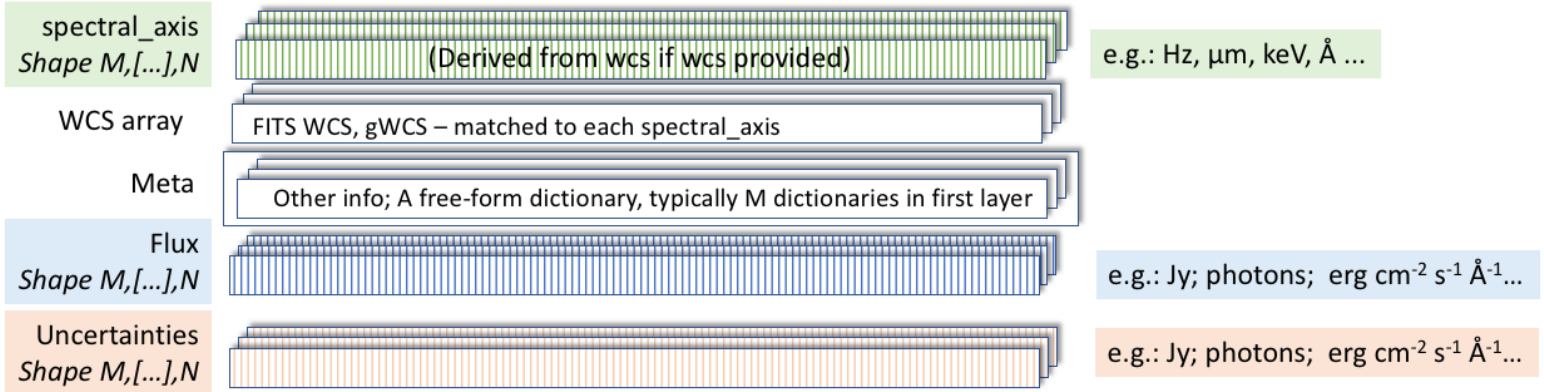
```
>>> from astropy.io import fits
>>> from astropy import units as u
>>> import numpy as np
>>> from matplotlib import pyplot as plt
>>> from astropy.visualization import quantity_support
>>> quantity_support()  # for getting units on the axes below  # doctest: +IGNORE_OUTP
```

Specutils data structures

Spectrum1D (one or many spectra with a single wavelength solution – N pixels in the wavelength dimension)



SpectrumCollection (M x ... spectra each of length N but different wavelength solutions)



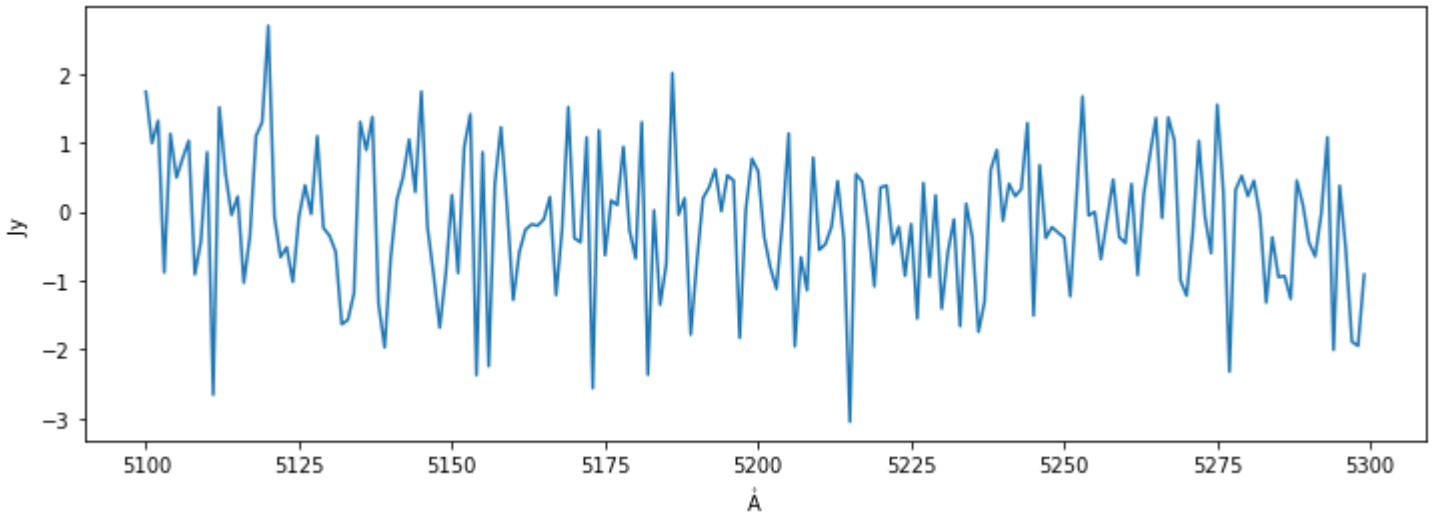
Creating spectra

Explicitly from arrays

`GWCS` object created implicitly

In [15]:
```
flux = np.random.randn(200)*u.Jy
wavelength = np.arange(5100, 5300)*u.AA
spec1d = Spectrum1D(spectral_axis=wavelength, flux=flux)

f, ax = plt.subplots()
ax.plot(spec1d.spectral_axis, spec1d.flux)
```

Out[15]:
```
[<matplotlib.lines.Line2D at 0x7f76d74e2048>]
```



Defined with a WCS

In [16]:
```
my_wcs = fitswcs.WCS(header={'CDELT1': 1, 'CRVAL1': 6562.8,
                             'CUNIT1': 'Angstrom',
                             'CTYPE1': 'WAVE',
                             'RESTFRQ': 1400000000, 'CRPIX1': 25})
spec = Spectrum1D(flux=[5,6,7] * u.Jy, wcs=my_wcs)
spec.wavelength

spec.wcs.pixel_to_world(np.arange(3))
```

Out[16]: $[6.5388 \times 10^{-7},\ 6.5398 \times 10^{-7},\ 6.5408 \times 10^{-7}]\ \mathrm{m}$

- Uses adapter class paradigm for choosing appropriate WCS interface
- Move to APE14 ✨in progress✨ (i.e. to be worked on during hacks)

`SpectralRegion`: easy region support

- Supports arithmetic for sub region selections
- Simple invert operations
- Analysis functions support optional inclusion

In [17]:

```
sr = (SpectralRegion(0.15*u.um, 0.2*u.um) +
      SpectralRegion(0.3*u.um, 0.4*u.um) +
      SpectralRegion(0.45*u.um, 0.6*u.um) +
      SpectralRegion(0.8*u.um, 0.9*u.um) +
      SpectralRegion(1.0*u.um, 1.2*u.um) +
      SpectralRegion(1.3*u.um, 1.5*u.um))

# Get on spectral region (returns a SpectralRegion instance)
print(sr[0])

# Slice spectral region.
print(sr[3:5])
```

```
Spectral Region, 1 sub-regions:
  (0.15 um, 0.2 um)
Spectral Region, 2 sub-regions:
  (0.8 um, 0.9 um)   (1.0 um, 1.2 um)
```

Region extraction

In [18]:
```
region = SpectralRegion(8*u.nm, 22*u.nm)
spectrum = Spectrum1D(spectral_axis=np.arange(1, 50) * u.nm, flux=np.random.sam
ple(49)*u.Jy)
sub_spectrum = extract_region(spectrum, region)

sub_spectrum.spectral_axis
```

Out[18]: $[8,\ 9,\ 10,\ 11,\ 12,\ 13,\ 14,\ 15,\ 16,\ 17,\ 18,\ 19,\ 20,\ 21,\ 22]\ \mathrm{nm}$

Analysis functions

- Centroiding
- Equivalent widths
- Line flux
- SNR

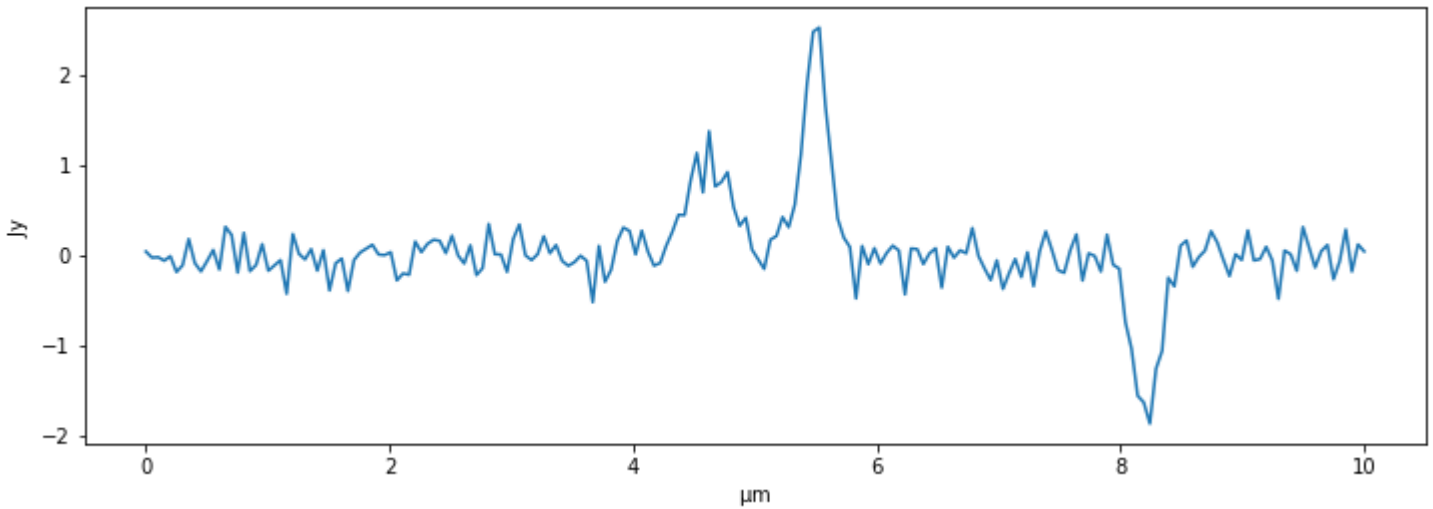
Simple manipulations

- Smoothing
- Uncertainty estimations

Spectral fitting and line finding

In [19]:
```
g1 = models.Gaussian1D(1, 4.6, 0.2)
g2 = models.Gaussian1D(2.5, 5.5, 0.1)
g3 = models.Gaussian1D(-1.7, 8.2, 0.1)
x = np.linspace(0, 10, 200)
y = g1(x) + g2(x) + g3(x) + np.random.normal(0., 0.2, x.shape)
spectrum = Spectrum1D(flux=y*u.Jy, spectral_axis=x*u.um)

plt.plot(spectrum.spectral_axis, spectrum.flux)
```

Out[19]:
```
[<matplotlib.lines.Line2D at 0x7f76bccde2e8>]
```



In [20]:

```
noise_region = SpectralRegion(0*u.um, 3*u.um)
spectrum = noise_region_uncertainty(spectrum, noise_region)

lines = find_lines_threshold(spectrum, noise_factor=3)

print(lines)
```

```
   line_center     line_type  line_center_index
        um
------------------ ---------- -----------------
 4.623115577889448   emission                92
 5.527638190954774   emission               110
3.6683417085427137 absorption                73
 8.241206030150755 absorption               164
```

Model parameter estimation

- Some pre-built initializers
- Support for custom initilizers (a bit wonky, involves settings to private variable)

In [21]:
```
sub_region = SpectralRegion(4*u.um, 5*u.um)
sub_spectrum = extract_region(spectrum, sub_region)

print(estimate_line_parameters(sub_spectrum, models.Gaussian1D()))
```

```
Model: Gaussian1D
Inputs: ('x',)
Outputs: ('y',)
Model set size: 1
Parameters:
        amplitude             mean              stddev
            Jy                 um                 um
    ------------------ ----------------- ------------------
    1.3735534091293726 4.614116745803695 0.3517587939698492
```

In [22]:

```
sub_region = SpectralRegion(4*u.um, 5*u.um)
sub_spectrum = extract_region(spectrum, sub_region)

mh = models.MexicanHat1D()
estimators = { 'amplitude': lambda s: max(s.flux),
               'x_0': lambda s: centroid(s, region=None),
               'stddev': lambda s: fwhm(s) }
mh._constraints['parameter_estimator'] = estimators

print(estimate_line_parameters(spectrum, mh))
```

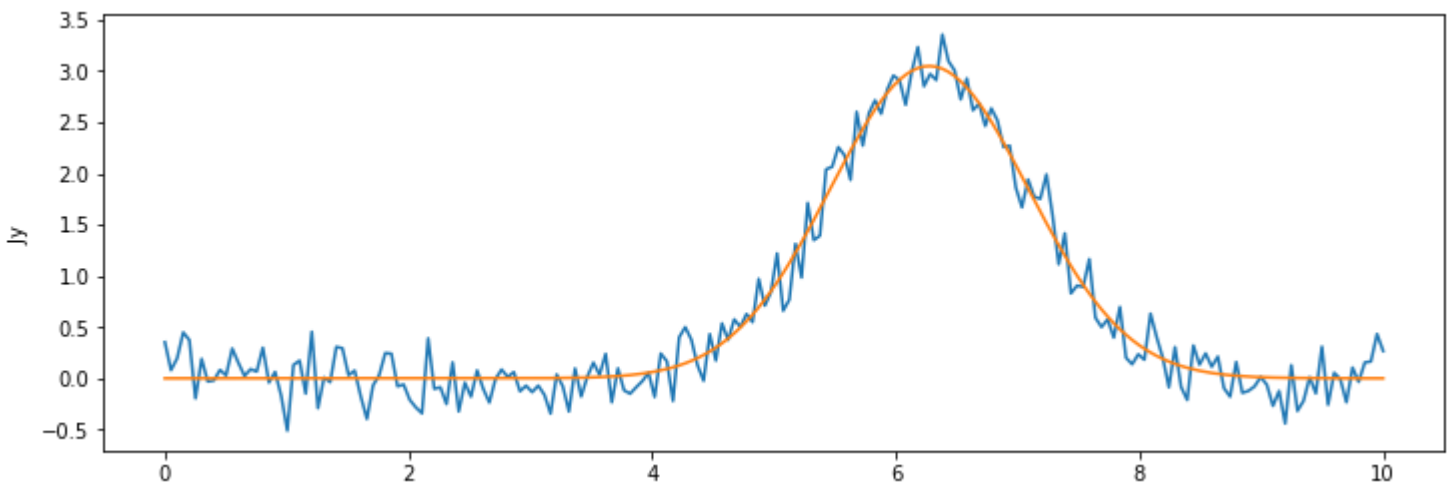
```
Model: MexicanHat1D
Inputs: ('x',)
Outputs: ('y',)
Model set size: 1
Parameters:
        amplitude             x_0        sigma
            Jy                 um
    ------------------ ----------------- -----
    2.5216847113707703 1.960913169140114   1.0
```

Line fitting

- Simple interface to fitting machinery
- Allows for individual model or compound model fitting
- Allows for window sizes or specific regions (backporting `SpectralRegion` in progress)

In [23]:
```
np.random.seed(0)
x = np.linspace(0., 10., 200)
y = 3 * np.exp(-0.5 * (x- 6.3)**2 / 0.8**2)
y += np.random.normal(0., 0.2, x.shape)
spectrum = Spectrum1D(flux=y*u.Jy, spectral_axis=x*u.um)

# Fit the spectrum and calculate the fitted flux values (``y_fit``)
g_init = models.Gaussian1D(amplitude=3.*u.Jy, mean=6.1*u.um, stddev=1.*u.um)
g_fit = fit_lines(spectrum, g_init)
y_fit = g_fit(x*u.um)

# Plot the original spectrum and the fitted.
plt.plot(x, y)
plt.plot(x, y_fit)
```

Out[23]:
```
[<matplotlib.lines.Line2D at 0x7f76bcaccf98>]
```

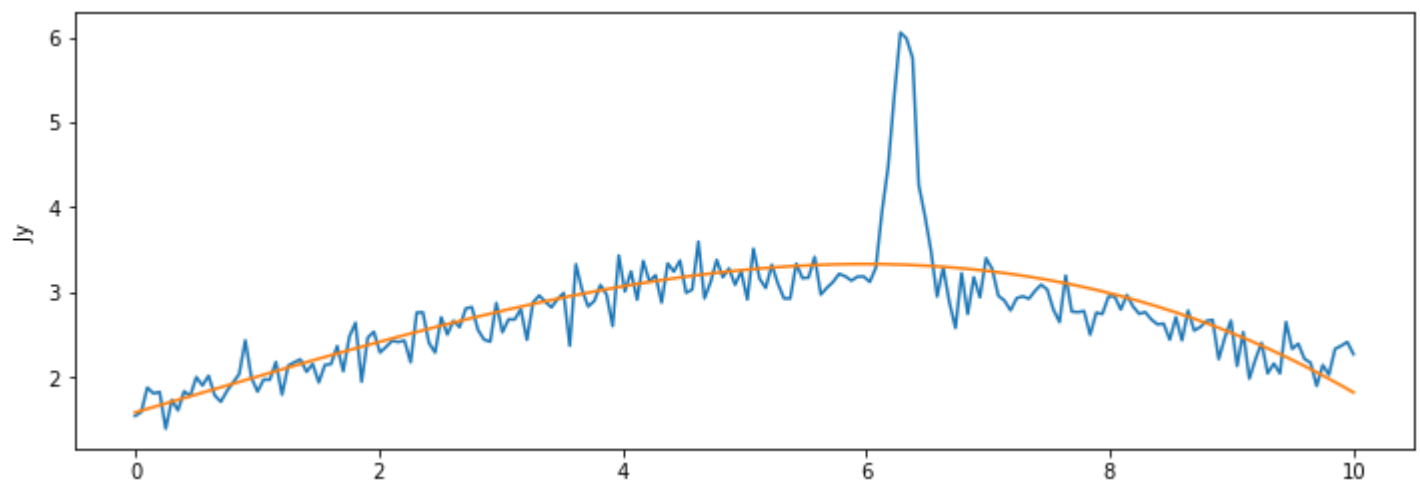


Continuum fitting

- Generic functionality to ignore spikes via smoothing before attempting continuum fitting
- Functional formalism for accessing the fitting machinery

In [24]:
```
x = np.linspace(0., 10., 200)
y = 3 * np.exp(-0.5 * (x - 6.3)**2 / 0.1**2)
y += np.random.normal(0., 0.2, x.shape)

y_continuum = 3.2 * np.exp(-0.5 * (x - 5.6)**2 / 4.8**2)
y += y_continuum

spectrum = Spectrum1D(flux=y*u.Jy, spectral_axis=x*u.um)
g1_fit = fit_generic_continuum(spectrum)
y_continuum_fitted = g1_fit(x*u.um)

plt.plot(x, y)
plt.plot(x, y_continuum_fitted)
```

Out[24]:
```
[<matplotlib.lines.Line2D at 0x7f76bcc4cdd8>]
```

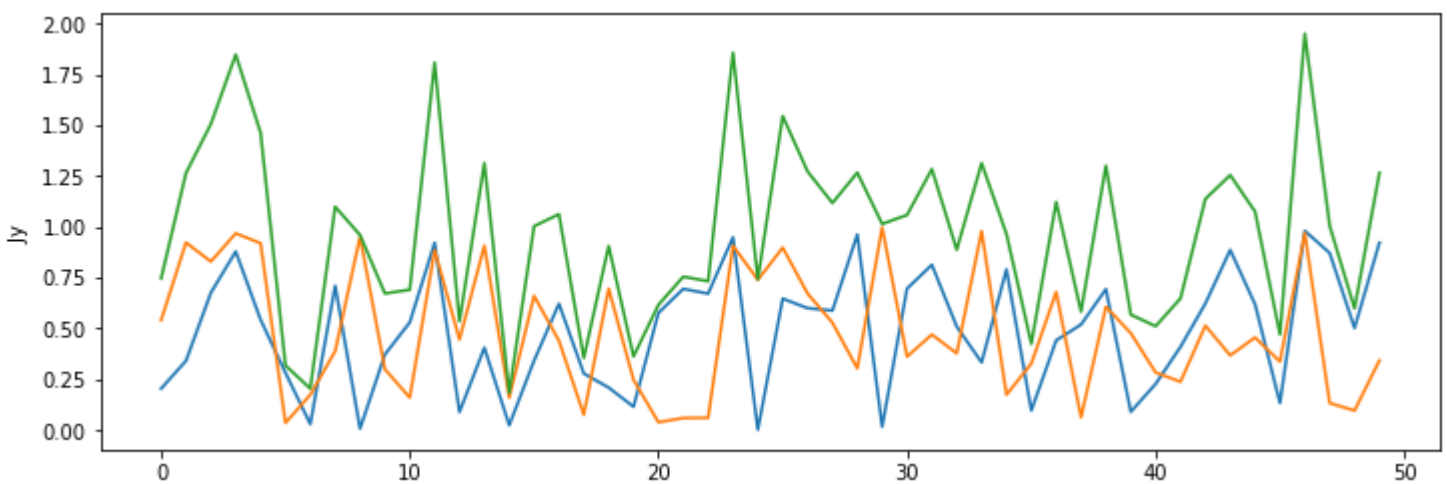


Spectral arithmetic

`NDData` subclass, supports basic arithmetic tasks with WCS and uncertainty propagation

In [25]:
```
spec1 = Spectrum1D(spectral_axis=np.arange(50)*u.nm, flux=np.random.sample(50)*
u.Jy)
spec2 = Spectrum1D(spectral_axis=np.arange(50)*u.nm, flux=np.random.sample(50)*
u.Jy)
spec3 = spec1 + spec2

f, ax = plt.subplots()
ax.plot(spec1.flux); ax.plot(spec2.flux); ax.plot(spec3.flux)
```

Out[25]:
```
[<matplotlib.lines.Line2D at 0x7f76bca252e8>]
```



Custom data loading/writing

- Leverage the astropy io machinery
- Picks up custom loading functions in user directories (to be extended in the future)

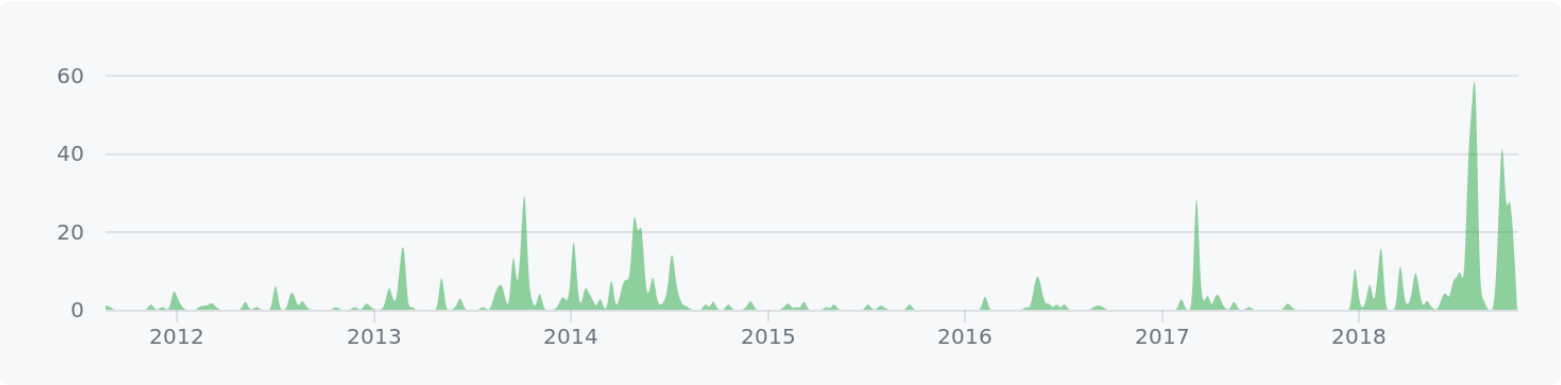
In [26]:

```
def identify_generic_fits(origin, *args, **kwargs):
    return (isinstance(args[0], str) and
            os.path.splitext(args[0].lower())[1] == '.fits')


@data_loader("my-format", identifier=identify_generic_fits,
             extensions=['fits'])
def generic_fits(file_name, **kwargs):
    name = os.path.basename(file_name.rstrip(os.sep)).rsplit('.', 1)[0]

    with fits.open(file_name, **kwargs) as hdulist:
        header = hdulist[0].header

        tab = Table.read(file_name)

        meta = {'header': header}
        wcs = WCS(hdulist[0].header)
        uncertainty = StdDevUncertainty(tab["err"])
        data = tab["flux"] * Unit("Jy")

    return Spectrum1D(flux=data, wcs=wcs, uncertainty=uncertainty, meta=meta)
```

```
---------------------------------------------------------------------------
IORegistryError                           Traceback (most recent call last)
<ipython-input-26-1a77de4856ab> in <module>
      5
      6 @data_loader("my-format", identifier=identify_generic_fits,
----> 7              extensions=['fits'])
      8 def generic_fits(file_name, **kwargs):
      9     name = os.path.basename(file_name.rstrip(os.sep)).rsplit('.', 1)[0
]
```

Growing community

Oct 2, 2011 – Dec 4, 2018

Contributions: **Commits**

Contributions to master, excluding merge commits